

AXIOM
SPACE

INTELLIGENT LEARNING AGENT SYSTEM

03

DOC 03

2026 年软件杯 · A3 组

系统设计与开发说明书

System Design & Development Specification

完整说明系统架构、智能体设计、核心实现、系统集成、优化与工程边界。

项目：AXIOM Space

参赛队伍：给春稗以秋实

文档编号：03

版本：提交版 · 2026.07

DOCUMENT 03 / REVIEWER GUIDE

文档导读

本册职责 完整说明系统架构、智能体设计、核心实现、系统集成、优化与工程边界。

文档信息

文档编号：03

文档性质：系统设计与开发实现说明

核心职责：回答“AXIOM Space 如何把需求变成可以运行、可以追溯的多智能体学习系统？”

上游文档：[02-软件需求规格说明书](#)、[10-产品创新与设计决策全记录](#)

下游文档：[04-测试说明书与测试报告](#)、[05-部署说明书](#)

目录

Word 中可点击条目直接跳转；目录层级与正文书签保持一致。

- 01 技术方案总览
- 02 1. 设计目标与开发原则
- 03 2. 总体架构
- 04 3. 核心运行闭环
- 05 4. 多智能体与 Agent Harness
- 06 5. 领域对象与数据事实
- 07 6. 动态画像与教学策略
- 08 7. 路径、资源、推送与评估
- 09 8. RAG 与知识图谱
- 10 9. UI、系统集成与工程实现
- 11 10. 可信、安全、性能与边界
- 12 11. 需求到实现的追溯
- 13 12. 对象真源与设计决策索引

- 14 附录 A：领域对象与契约真源
 - 15 附录 B：开发技术规范真源
 - 16 附录 C：产品创新与设计决策索引
 - 17 资料真源
-

技术方案总览

AXIOM Space 的技术创新不在于“接入了一个大模型”，而在于用 **Agent Harness** 把模型的发散能力牵引到同一个可验证学习对象上。

结构示意图

课程资料 / 学习主题

- 动态画像识别“这个学生是谁、目前卡在哪里”
- Learn 把目标组织成可执行路径
- Forge 将对话绑定具体卡片和学习步骤
- Agent A 负责教学与追问
- Agent B 负责观察、画像、卡片和证据
- 子 Agent 按需完成审核、资源、评估等隔离任务
- 用户自己的输出进入卡片
- 审核与评估区分“写清楚”和“真正掌握”
- RAG、图谱、路径、画像和推送共同决定下一步

技术问题	AXIOM Space 的设计决定	对学生的直接价值
通用 Agent 输出停在回复	自研 Harness 绑定入口、上下文、角色、工具、证据、反馈和知识	一次对话能改变卡片、画像、路径、资源和图谱
多 Agent 容易变角色表演	双 Agent 底座负责前台教学与后台观察，子 Agent 按需隔离	教学、记录和审核彼此制约，过程可追踪
画像容易变静态标签	来源、置信度、历史快照和教学策略编译	画像会改变下一问、资源、路径或推送
向量库容易成为第二事实源	Postgres 保存事实，RAG 只是可重建派生索引	召回旧知识但不污染真实卡片
图谱容易成为 3D 装饰	Card / Edge / Cluster 契约与多种认知投影	同一证据可从结构、任务、来源和掌握角度查看
AI 生成容易伪造掌握	用户输出、卡片准入、独立审核、迁移评估分层	“AI 说过”不会被当成“学生会了”
多模态生成耗时且脆弱	状态机、进度事件、失败隔离、类型校验和预览	用户知道系统在做什么，部分失败不吞掉成功结果

1. 设计目标与开发原则

1.1 设计目标

系统设计围绕一个可否证目标展开：学生完成一次学习后，系统不仅保存 AI 的回答，还要保存学生自己的输出、为什么通过或未通过、下一步如何变化，以及下次如何继续。

设计必须同时满足：

- 对话式不少于 6 维动态画像；
- 真实多智能体协同；
- 至少 5 类个性化资源；

- 动态学习路径与资源推送；
- 智能辅导和学习效果评估；
- 防幻觉、内容安全、进度与稳定运行；
- 完整课程资料输入和长期学习状态。

1.2 核心开发原则

1. 依赖从外指向内。核心业务不依赖 Next.js、Hono、Prisma 或具体模型 SDK。
2. 数据库是事实源。缓存、RAG、向量索引和页面读模型都是可重建派生层。
3. 所有 AI 动作落到业务对象。回复本身不是业务完成。
4. 错误显式暴露。部分成功、失败、重试和降级具有不同状态。
5. 用户输出优先。AI 不把自己的内容冒充成用户掌握。
6. 清晰、准确、必要。长期知识必须对象清楚、有依据、通过删除测试。
7. 类型与契约统一。前后端通过 Hono RPC 和共享领域类型减少漂移。
8. 页面展示证据。架构能力最终必须变成用户可见的状态、理由和结果。

2. 总体架构

2.1 技术栈

层次	技术与职责
表现层	Next.js App Router、React、Tailwind CSS、shadcn/ui
状态与数据请求	React Query 管理服务端数据，Zustand 管理全局 UI 状态
API 网关	Hono 路由、参数验证、统一错误、RPC 类型推导
核心业务	纯 TypeScript 领域对象、Agent、画像、学习、RAG、推送、评估
基础设施	Prisma、PostgreSQL / SQLite、文件存储、模型与 RAG 适配器
认证	Better Auth
异步与事件	后台任务、通知总线、资源进度事件、状态查询
AI 与检索	DeepSeek / OpenAI-compatible 模型调用、讯飞星火 WebSocket 适配、Agent Runtime、Qdrant 语义索引与 LightRAG 图增强检索

2.2 分层结构

结构示意图



```
↓ 校验、鉴权、错误转换、转发
server/core
↓ 领域对象、业务规则、Agent、学习、评估、RAG 契约
server/infra
↓ 数据库、文件系统、模型、向量库、外部服务适配
```

核心法则是：内层不认识外层。页面不能直接承担领域规则；API 不能把业务逻辑堆在路由中；核心域不能依赖框架实现。

2.3 数据流

结构示意

```
页面
→ 自定义 Hook
→ Hono RPC Client
→ Hono API
→ 领域服务 / Agent Runtime
→ Prisma 事实数据
→ 可选派生索引、文件、模型与后台任务
→ 事件或查询刷新页面
```

3. 核心运行闭环

3.1 资料导入

结构示意

```
课程资料
→ 文档解析
→ 概念与来源提取
→ literature / fleeting 卡片
→ WikiLink / Edge / Cluster
→ 学习路径
→ RAG 索引任务
```

资料导入不是收藏。它必须把外部内容变成可学习对象，并保留失败边界：某个后续步骤失败时，已经合法写入的卡片不能被无提示抹掉。

3.2 学习任务进入 Forge

结构示意

```
LearningPathStep
→ 绑定或创建 Card
→ 创建或恢复 LearningSession
→ 构建 Current Focus
→ 装配画像、旧知识、路径、资源和证据上下文
→ Agent A 开始教学
```

3.3 一轮对话

结构示意图

用户消息

- API 鉴权与 Vault 边界
- Agent Runtime 组合根
- ContextBuilder
- Prompt / Policy
- 模型流式输出与工具调用
- 消息持久化
- Agent B 后台分析
- 画像、观察、卡片、推送或事件更新

3.4 掌握闭环

结构示意图

学生原始表达

- 主动追问与换角度解释
- 学生修改卡片
- 清晰 / 准确 / 必要审核
- 若失败：保留原因并继续学习
- 若通过：允许进入长期知识
- 陌生迁移或正式评估
- 更新掌握证据、画像和路径

4. 多智能体与 Agent Harness

4.1 Harness 的工作定义

Harness 不是一个大而全的 Agent 类，而是七类牵引机制的组合：

9. 入口：当前 Vault、课程、路径、步骤和卡片。
10. 上下文：用户画像、对话、旧卡、检索引用、资源和评估。
11. 角色：Agent A、Agent B 和按需子 Agent。
12. 工具：有类型、有风险等级、有确认边界的业务能力。
13. 证据：来源、用户原话、对象标识、评估和审计记录。
14. 反馈：页面事件、进度、失败、建议和用户确认。
15. 知识：数据库事实、个人卡片、课程资料和派生索引。

模型仍可以生成多样输出，但 Harness 决定它看到什么、能做什么、何时写入、写入哪里、如何被验证。

4.2 双 Agent 底座

- **Agent A: 前台教学 Agent。** 负责围绕当前学习对象解释、提问、追问、提示和组织用户输出。
- **Agent B: 后台分析 Agent。** 负责从真实对话中提取观察、更新画像候选、同步卡片线索和生成后续干预建议。

双 Agent 的价值不是“两个聊天头像”，而是将教学和记录分开：正在说服用户的 Agent 不应同时不受约束地证明自己教学成功。

4.3 子 Agent

子 Agent 按任务需要隔离，例如：

- 卡片质量审核；
- 资源设计与生成；
- 评估与量规判断；
- 关系发现；
- 事实或来源检查。

子 Agent 不需要全部常驻，也不需要页面上被包装成固定角色。它们的价值是隔离目标、上下文、工具和失败。

4.4 工具治理

工具是模型影响系统的唯一受控边界。每项工具需要定义：

- 输入与输出；
- 所属 Vault 和对象；
- 幂等与重复调用处理；
- 只读、写入、删除或外部调用风险；
- 是否需要用户确认；
- 失败时的错误和审计记录。

4.5 多供应商模型适配

Agent Harness 通过统一模型配置与适配层隔离供应商差异。当前运行基线采用 DeepSeek 的 OpenAI-compatible 接口；科大讯飞星火已完成 WebSocket 工程接入，可按部署方案启用。

讯飞适配层已经实现：

- v1.1、v2、v3.5 三个协议版本；
- 基于 HMAC-SHA256 的服务端鉴权；
- 统一消息到星火请求结构的转换；

- 分片聚合、流式回调和 Token 用量读取；
- API 错误、连接错误与超时控制；
- 登录保护的配置检查与在线兼容性探测接口。

模型供应商切换不改变 Card、Path、Profile、Assessment 等领域对象，也不绕过工具 schema、用户确认、Vault 隔离和审计机制。完整的 AI 工具与研发流程披露见 [09-AI 工具使用与 AI 辅助开发流程说明](#)。

5. 领域对象与数据事实

5.1 核心聚合

- Vault：个人知识与学习边界。
- Card：知识对象；区分 fleeting、literature、permanent。
- LearningPath：学习任务组。
- LearningPathStep：可执行任务。
- LearningSession / LearningMessage：围绕对象的学习过程。
- EducationProfile / Observation / ProfileSnapshot：画像事实、来源和历史。
- Assessment / Mastery / Evidence：学习结果与依据。
- Resource / PushRecord：学习资源与推送决策。
- RagDocumentIndex / RagReference：派生索引与真实引用。
- Cluster / Edge / WikiLink：知识结构和关系。
- BackgroundJob / Notification / AuditLog：异步执行、页面反馈和可追踪性。

5.2 三类卡片

类型	含义	主要来源	是否代表掌握
fleeting	零散想法、疑问、尚未完成的理解	用户表达、学习过程	否
literature	课程、文献、AI 资源或外部资料	外部来源	否
permanent	通过长期知识准入审核的用户理解	用户输出与审核	只代表内容准入，不自动代表全部能力掌握

5.3 事实与派生层

- Prisma 数据库保存用户、卡片、路径、会话、画像、评估和资源事实。
- RAG、向量索引、隐藏关系和页面统计是可重建派生数据。

- 派生层可以提供建议，不能反向篡改事实对象。
- 索引失败与事实保存失败必须是两个不同状态。

6. 动态画像与教学策略

6.1 画像不是六个分数

画像至少包含知识基础、目标、表达或认知偏好、易错点、节奏、资源偏好等维度，但每个维度还应包含：

- 当前判断；
- 来源证据；
- 置信度；
- 观察时间；
- 是否与既有判断冲突；
- 是否需要进一步验证。

6.2 两条画像通道

- **即时轻量通道：** 从当前对话形成候选观察，为下一问提供低成本调整。
- **后台深度通道：** 汇总多轮消息、卡片、评估和行为，生成更稳定的画像更新。

6.3 从画像到教学行动

画像不能停在 **Cognition** 展示页。它需要被压缩为教学策略，例如：

- 先预测再解释；
- 使用代码、图示、反例或类比；
- 降低或提高任务步长；
- 召回某张旧卡；
- 跳过已掌握前置步骤；
- 推送某类资源并说明原因。

7. 路径、资源、推送与评估

7.1 路径

路径由目标、前置关系、当前画像、已有卡片和掌握证据共同决定。路径调整需要保留原因，避免用户只看到顺序变化却不知道为什么。

7.2 资源

资源编排从“当前缺口和用途”开始，而不是从“生成一个文件”开始。不同资源类型使用不同的提示、结构校验和渲染方式，并通过状态机与进度事件反馈。

7.3 推送

推送机制先由多个查找器寻找候选，再由 AI 审核是否相关、必要和适合当前用户。系统建议与真实创建分离：先说明原因，用户确认后再生成或加入任务。

7.4 三层评估

16. 费曼式对话评估：用户能否用自己的话讲清机制。

17. 卡片准入审核：内容是否清晰、准确、必要，是否有反例、边界和关联。

18. 能力与学习健康评估：用户能否在陌生任务中迁移，长期表现是否稳定。

三层不能被一个总分抹平。卡片通过不等于能力掌握；当前高分也不等于长期保持。

8. RAG 与知识图谱

8.1 RAG 的职责

RAG 用于：

- 召回用户真实旧卡；
- 为当前对话提供来源；
- 发现相关但不等价的知识；
- 避免重复生成；
- 支持路径、资源、图谱和认知建议。

RAG 不负责保存用户事实，不把所有聊天流水无差别写入索引，也不以相似度冒充事实关系。

8.2 知识图谱的职责

图谱由 Card、Edge、Cluster 等稳定对象组成。同一份事实数据可以投影为不同视图：

- 总览：课程结构和知识分布；
- 邻域：当前知识与哪些旧知识相关；
- 任务流：下一步应该做什么；
- 证据：哪些资料、表达和评估支撑当前节点；
- 掌握：不同知识状态如何分布。

布局切换不是装饰变化，而是问题变化。

9. UI、系统集成与工程实现

9.1 页面职责

页面	单一职责
Landing / 入口	解释产品并进入个人学习空间
Dashboard	展示当前课程知识库和学习状态总览
Learn	生成、执行和调整学习路径
Forge	围绕当前学习对象完成对话、输出、资源和审核
Galaxy	从多种认知视角查看知识结构
Cognition	查看画像、观察、证据和下一步建议

9.2 类型化 RPC

所有 API 调用通过 Hono RPC 客户端与自定义 Hook 完成。页面不直接散落 fetch 或 axios。服务端通过验证器、统一错误和领域服务保持薄网关。

9.3 状态管理

- React Query: 服务端数据、失效和刷新。
- Zustand: 跨页面 UI 模式与当前选择。
- 组件本地状态: 仅用于局部交互。
- 领域状态: 保存在服务器事实对象中, 不由页面临时状态代替。

9.4 系统集成

系统集成围绕稳定契约完成:

- API 输入输出与领域对象映射;
- Agent 工具与业务服务;
- 数据库事务与派生任务;
- 事件通知与页面查询刷新;
- 资源清单与类型渲染器;
- RAG 引用与真实卡片回映射。

10. 可信、安全、性能与边界

10.1 可信生成与防幻觉机制

AXIOM Space 将概率生成结果置于可追溯、可校验、可复核的工程链路中：来源引用负责给出依据，结构校验负责约束输出形态，独立审核负责隔离生成与判定，工具确认负责控制写入动作，用户输出负责提供真实学习证据，审计记录负责还原关键过程。

该机制的工程目标是降低错误内容进入长期知识与掌握记录的概率，并为每项关键判断保留来源和复核入口。系统不以绝对化语言描述概率模型，而以证据完整性、行为边界和错误可恢复性建立可信度。

10.2 内容安全

外部资料、用户输入、模型输出和工具参数都需要边界检查。提示注入不得直接获得高风险工具权限，敏感数据不得写入日志、前端或导出。

10.3 性能和体验

- 流式输出尽早给出可见反馈。
- 资源状态区分排队、生成、校验、渲染、完成和失败。
- 多个资源可并行，但必须限制并发并隔离失败。
- 重资源可以先交付 HTML 或结构化预览，再异步完成最终媒体。
- 长任务不能用固定假进度掩盖真实等待。

10.4 适用边界与降级策略

- 系统运行状态以分项健康接口、执行日志和《04-测试说明书与测试报告》的复核记录为准。
- AI 评估输出掌握迹象、薄弱点与依据，并保留用户或教师复核入口。
- 外部模型、RAG 或渲染服务不可用时，系统采用明确错误、任务重试和派生能力降级；数据库事实与用户原始内容保持完整。
- 课程、画像和长期档案中的演示数据与真实用户研究数据分开管理，避免样例数据被解释为研究结论。
- 所有版本级能力认定统一回到需求编号、实现位置和测试证据，形成可审查的交付口径。

11. 需求到实现的追溯

需求	主要实现责任
FR-001	认证、Vault 边界、API 鉴权、Prisma 归属

需求	主要实现责任
FR-002	文档导入服务、卡片与关系、路径生成、索引任务
FR-003—FR-004	EducationProfile、Observation、画像协议、ContextBuilder
FR-005	LearningPath、Step、PathAdjustment、路径服务
FR-006—FR-008	Agent Harness、双 Agent、工具契约、Forge、卡片审核
FR-009—FR-011	ResourceGenerationOrchestrator、状态机、推送查找器与确认
FR-012	RAG 服务、索引身份、引用回映射
FR-013	Card / Edge / Cluster、Galaxy 视图与交互
FR-014—FR-016	教学策略、评估协议、Cognition 读模型
FR-017—FR-018	导出、事件、通知、后台任务与审计

12. 对象真源与设计决策索引

附录 A、B 完整保留领域对象契约与开发技术规范，作为系统实现的工程真源。围绕产品问题、候选方案、取舍依据、交互规则和创新形成过程的 10 份设计记录，统一归入《10-产品创新与设计决策全记录》，从而使本说明书专注回答“系统如何设计并实现”，设计决策记录专注回答“为什么这样设计”。

附录 A：领域对象与契约真源

06 对象模型与契约

1. 对象范围

这里说的“对象”，不是代码里的每一个 `class`，也不是页面组件。

这里说的是 AXIOM 这个产品里真实存在的领域对象：

- 用户看得见、能操作的对象。
- 系统必须长期保存的对象。
- 对象之间有明确归属、状态、生命周期和边界的对象。
- 即使以后换成桌面端、移动端或插件，也仍然会存在的对象。

数据库是最重要的参考。当前 `prisma/schema.prisma` 已经说明了很多真实对象：

结构示意图

```
user / account / session / verification
vault
card / cluster / edge / ragDocumentIndex
learningPath / learningPathStep
learningSession / learningMessage
vaultMemory / vaultCapability / vaultSkill
EducationProfileHistory / PathAdjustmentHistory / PushRecord
agentSession / agentAuditLog
```

在这些数据库对象之上，还存在一些产品层面的子对象和派生对象，例如：

- `fleeting / literature / permanent` 不是三张表，但它们是 `Card` 的三种领域子类型。
- `Cluster` 是卡片的星团分组对象。
- `LearningPath` 是任务路径对象。
- `LearningPathStep` 是路径里的单个任务对象。
- `Session metadata` 里保存的 `cardId / pathId / stepId / threadStatus` 是会话线程契约。
- `Cognition` 页面看到的画像、知识缺口、观察，是从多个底层对象聚合出来的派生对象。

所以本文档按“领域语义”来拆，不只照表名照抄。

2. 全量领域对象地图

2.1 身份与账户对象

对象	含义	当前实现
User	产品中的用户主体	<code>user</code>
AuthAccount	用户登录方式，如邮箱、第三方账号、密码凭据	<code>account</code>
AuthSession	登录态，用于判断当前请求属于谁	<code>session</code>

对象	含义	当前实现
VerificationToken	邮箱验证、登录验证等短期凭据	verification

这些对象不直接承载学习内容，但它们决定所有学习数据属于谁。

2.2 知识库对象

对象	含义	当前实现
Vault	用户的个人知识库，也是所有学习数据的边界	vault
VaultProfileCache	Vault 上的画像缓存，避免每次重新计算 Cognition	vault.profileCache

Vault 是最重要的根对象。Card、Cluster、Edge、Path、Session、Memory、Skill、Profile、PushRecord 都必须落在 Vault 或 User 下面。

2.3 卡片系统对象

对象	含义	当前实现
Card	知识沉淀的基本单位	card
FleetingCard	灵感卡、临时理解、正在打磨的知识	card.type = fleeting
LiteratureCard	文献卡、资料卡、外部来源记录	card.type = literature
PermanentCard	永久卡，已经被用户整理过、可长期复用	card.type = permanent
CardPath	卡片在 Vault 中的路径，如 permanent/概念.md	card.path
CardContent	卡片的 Markdown 内容	card.content
CardTitle	卡片标题	card.title
CardTags	卡片标签集合	card.tags JSON
CardClusterMembership	卡片属于哪个星团	card.clusterId
CardLinks	卡片的入链、出链、悬空链接	edge + WikiLink 解析
CardRagState	卡片是否进入 RAG 索引	ragDocumentIndex

Card 不是只有“三类”这么简单。它至少有五个维度：

维度	可能值	说明
类型维度	fleeting / literature / permanent	卡片在学习沉淀中的阶段
星团维度	属于某个 Cluster / 未分组	卡片在知识空间中的领域分组
任务维度	绑定某个 Step / 未绑定路径	卡片是否属于某条学习路径任务
图谱维度	有关系边 / 孤立节点 / 有悬空 WikiLink	卡片在知识网络中的连接情况
RAG 维度	pending / indexing / indexed / failed / disabled	卡片是否能被 AI 稳定召回

这意味着同一张卡片可以同时是：

结构示意图

- 一张 fleeting 卡
- 属于“操作系统”星团
- 绑定“进程调度学习路径”的第 2 步
- 通过 Edge 连接到“进程”和“线程”
- RAG 状态是 indexed

这些不是多个对象互相替代，而是同一个 Card 在不同维度上的状态。

2.4 星团与图谱对象

对象	含义	当前实现
Cluster	星团，知识领域分组，如“数据结构”“操作系统”	cluster
Edge	卡片之间的关系边	edge
EdgeType	关系类型	edge.type
WikiLink	Markdown 中的 [[概念名]]，可同步为 Edge	card.content + lib/wiki-links.ts
GalaxyNode	Galaxy 中展示的卡片节点	Card 派生
GalaxyEdge	Galaxy 中展示的关系连线	Edge 派生
GalaxyCluster	Galaxy 中展示的星团	Cluster 派生

Cluster 和 CardType 是两套完全不同的分类。

CardType 回答：

结构示意图

- 这张卡在学习沉淀中处于什么阶段？

Cluster 回答：

结构示意图

- 这张卡属于哪个知识领域？

所以一个 Cluster 里面可以同时有三类卡：

结构示意图

- Cluster：操作系统
 - > permanent：进程
 - > permanent：线程
 - > fleeting：时间片轮转的疑问
 - > literature：操作系统课程笔记

Edge 回答的是另一件事：

结构示意图

- 两张卡之间是什么关系？

当前已经出现或应被承认的关系类型包括：

类型	含义
related	一般相关
prerequisite	前置依赖
derived	推导、派生
counter	对比、反例
wikilink	由 Markdown WikiLink 自动同步出的链接
suggests	推荐学习关系，部分工具层已有语义
extends / extension	扩展关系，部分工具层已有语义
contrast	对比关系，部分工具层已有语义
application	应用关系，推荐工具中已有语义
analogy	类比关系，推荐工具中已有语义

数据库没有 `enum` 限制，所以文档里要把这些关系收敛成清楚的领域枚举。

2.5 学习任务对象

对象	含义	当前实现
LearningPath	学习路径，也可以理解为任务组	<code>learningPath</code>
LearningPathStep	路径中的一个学习任务	<code>learningPathStep</code>
PathTopic	路径主题	<code>learningPath.topic</code>
PathDifficulty	路径难度	<code>learningPath.difficulty</code>
PathSource	路径来源，如 AI、图谱、手动、导入资料	<code>learningPath.source</code>
PathStatus	路径状态	<code>learningPath.status</code>
PathProgress	路径进度	<code>totalSteps / doneSteps</code>
StepOrder	步骤顺序	<code>learningPathStep.order</code>
StepConcept	步骤对应概念	<code>learningPathStep.concept</code>
StepChapter	步骤所属章节或概念组	<code>learningPathStep.chapter</code>
StepStatus	步骤状态	<code>learningPathStep.status</code>
StepMastery	步骤掌握度	<code>learningPathStep.mastery</code>
StepPrerequisites	步骤前置依赖	<code>learningPathStep.prerequisites</code> JSON
PathAdjustment	路径调整建议或调整记录	<code>PathAdjustmentHistory.adjustment</code> JSON
LearningStage	路径调整引擎里的阶段对象	<code>server/core/learning/path-adjustment-engine.ts</code>

`LearningPath` 是“任务层”的对象，不是卡片的另一种分类。

一条路径可以绑定多张卡：

结构示意

```
LearningPath: 第一性原理入门
Step 1: 什么是第一性原理 -> Card A
Step 2: 第一性原理 vs 类比思维 -> Card B
Step 3: 编程中的第一性原理 -> Card C
```

同一张 Card 也可以被多个路径复用。例如“递归”这张卡，既可能出现在“数据结构入门”路径，也可能出现在“算法复杂度”路径里。

2.6 Forge 与会话对象

对象	含义	当前实现
LearningSession	Forge / Agent 学习会话	learningSession
LearningMessage	会话中的消息	learningMessage
SessionKind	会话类型，如普通聊天、卡片线程	learningSession.metadata.sessionKind
ThreadMetadata	线程绑定上下文	learningSession.metadata.JSON
ThreadStatus	线程状态，如 active / archived	learningSession.metadata.threadStatus
MessageRole	消息角色	learningMessage.role
AgentSession	旧 Agent 会话存储，替代 .axiom/sessions	agentSession
AgentAuditLog	Agent 审计日志	agentAuditLog

Forge 里的会话不是单纯聊天记录。它可能绑定：

结构示意

```
普通会话：
  sessionKind = conversation

卡片线程：
  sessionKind = card-thread
  cardId = 某张 Card

路径步骤线程：
  cardId = 某张 Card
  pathId = 某条 LearningPath
  stepId = 某个 LearningPathStep
```

所以 LearningSession.metadata 虽然现在只是 JSON 字符串，但它在领域上是一个很重要的对象，应该被显式建模。

2.7 记忆、能力、技能与画像对象

对象	含义	当前实现
VaultMemory	Vault 级长期记忆	vaultMemory
MemoryCategory	记忆分类：preference / style / context / fact	vaultMemory.category
VaultCapability	用户对某个概念的能力状态	vaultCapability

对象	含义	当前实现
CapabilityStatus	known / learning / mastered	<code>vaultCapability.status</code>
VaultSkill	用户表现出来的技能	<code>vaultSkill</code>
SkillEvidence	技能证据	<code>vaultSkill.evidence</code>
EducationProfile	六维学习画像	<code>EducationProfileHistory.profile</code> JSON
DimensionScore	某一画像维度的分数、置信度和证据	<code>server/core/learning/education-profile.ts</code>
EducationProfileHistory	画像历史快照	<code>EducationProfileHistory</code>
CognitionData	Cognition 页面聚合出来的学习反馈	API 派生
KnowledgeGap	知识缺口	Cognition 派生
Observation	系统观察	Cognition 派生

这些对象回答：

结构示意图

系统通过用户的学习行为，已经认识了用户什么？

它们不能替代 Card 和 Path，但会影响后续路径生成、资源推荐和学习反馈。

2.8 资源与推送对象

对象	含义	当前实现
PushRecord	一次资源推送记录	<code>PushRecord</code>
PushableResource	可推送资源	<code>PushRecord.resources</code> JSON
ResourceType	资源类型，如 document / mindmap / quiz / code / video / ppt	<code>ResourceGenerationState.ts</code>
ResourceGenerationEntry	某类资源的生成状态	<code>ResourceGenerationState.ts</code>
ResourceProgress	资源生成进度	Agent SSE / 前端展示
GeneratedResourceItem	Forge 中展示的生成资源项	<code>components/resources/resource-cards.tsx</code>
ResourceManifestItem	literature 卡片关联的资源清单项	<code>vault.ts / forge-editor.tsx</code>

资源不是 Card 的替代物。

更准确的关系是：

结构示意图

Card 是知识对象。

Resource 是围绕某个学习目标或卡片生成的辅助材料。

PushRecord 是系统把资源推荐给用户的一次记录。

资源最终可以保存成 literature 卡，也可以作为某张卡的补充材料。

2.9 RAG 与检索对象

对象	含义	当前实现
RagDocumentIndex	Card 在 RAG 系统里的索引状态	ragDocumentIndex
RagWorkspace	某个 Vault 对应的 LightRAG 工作区	workspace
RagDocumentId	Card 在 RAG 中的文档 ID	documentId
RagContentHash	当前索引内容的 hash	contentHash
RagTrackId	后台索引任务 ID	trackId
RagReference	AI 回复时引用的卡片或文献	Agent / RAG 返回
RagQueryContext	RAG 查询结果上下文	server/core/rag/lightrag-service.ts

RAG 是 sidecar，不是源数据。

源数据仍然是 Card。

结构示意

Card 保存成功 != RAG 索引成功

这两个状态要分开理解。

2.10 前端承载对象

这些不是数据库对象，但它们是产品交互里的对象。

对象	含义	当前实现
AppMode	当前模式：dashboard / forge / galaxy / cognition / learn	stores/mode-store.ts
SelectedNode	当前选中的卡片节点	stores/mode-store.ts
SelectedPath	当前选中的学习路径	selectedPathId
ActiveLearningStep	当前正在学习的步骤	activeLearningStepId
PanelLayout	Forge 面板布局	panelLayout
GraphLayoutMode	Galaxy 布局模式	graphLayoutMode
SessionSummary	Forge 侧边栏会话摘要	stores/agent-store.ts
AppNotification	前端通知	hooks/use-notifications.ts
DashboardStats	Dashboard 聚合统计	types/dashboard.ts
RecentActivity	最近活动	types/dashboard.ts
GrowthPoint	成长曲线上的一个点	types/dashboard.ts

它们负责“用户当前看到什么、选中了什么”，但不能替代后端对象关系。

2.11 DDD 视角下还要承认的对象

如果按 DDD 来看，前面列出的很多对象属于实体；但领域模型不只有实体，还应该包含值对象、聚合、领域服务、领域事件和过程对象。

这些对象不一定都有单独的数据表，但它们在业务上真实存在。

2.11.1 值对象

值对象没有独立生命周期，通常依附在实体上。

值对象	依附对象	含义
CardType	Card	fleeting / literature / permanent
CardPath	Card	卡片在 Vault 内的唯一路径
MarkdownContent	Card	卡片正文内容
CardTitle	Card	卡片标题
CardTags	Card	标签集合
ClusterColor	Cluster	星团颜色
ClusterPosition	Cluster	星团排序
EdgeType	Edge	related / prerequisite / derived 等关系类型
EdgeWeight	Edge	关系强度
LearningPathStatus	LearningPath	active / completed / archived
LearningPathSource	LearningPath	ai / graph / manual / import-document
Difficulty	LearningPath / Resource	beginner / intermediate / advanced
StepStatus	LearningPathStep	locked / available / learning / completed / mastered
MasteryScore	LearningPathStep / Capability	掌握度分数
EstimatedMinutes	LearningPathStep / Resource	预计耗时
PrerequisiteSet	LearningPathStep	前置依赖集合
SessionKind	LearningSession	conversation / card-thread / path-step-thread
ThreadStatus	LearningSession	active / archived
MessageRole	LearningMessage	system / user / assistant / tool_result
DimensionScore	EducationProfile	score + confidence + evidence
CapabilityStatus	VaultCapability	known / learning / mastered
MemoryCategory	VaultMemory	preference / style / context / fact
PushTrigger	PushRecord	stage_completion / assessment_pass / low_dimension / scheduled
ResourceType	PushableResource	document / mindmap / quiz / code / video / ppt
RagSyncStatus	RagDocumentIndex	pending / indexing / indexed / failed / disabled
ContentHash	RagDocumentIndex	RAG 内容 hash

这些值对象现在很多还是字符串或 JSON。后续应该收敛成共享类型。

2.11.2 聚合与聚合根

聚合回答的是“哪些对象应该被一起维护一致性”。

聚合根	聚合内对象	需要保护的规则
User	AuthAccount、AuthSession、Vault 引用	用户只能访问自己的数据
Vault	Card、Cluster、Edge、Path、Session、Memory、Capability、Skill 的归属边界	不能跨 Vault 读写、关联、检索
Card	CardType、CardPath、Content、Tags、RagDocumentIndex 引用	同 Vault 内路径唯一；升级 permanent 后线程归档
KnowledgeGraph	Card 节点、Edge、Cluster	Edge 两端必须属于同一 Vault；删除 Card 清理 Edge
LearningPath	LearningPathStep、PathAdjustmentHistory	Step 必须属于 Path；Path 进度来自 Step
LearningSession	ThreadMetadata、LearningMessage	Message 必须属于 Session；归档线程不能继续写入
CognitionProfile	VaultMemory、VaultCapability、VaultSkill、EducationProfileHistory	画像必须来自证据，不能凭空生成
ResourcePush	PushRecord、PushableResource、用户反馈	推送必须有触发原因、过期时间和反馈记录
RagIndex	RagDocumentIndex、RagReference	RAG 是 Card 的检索副本，不是源数据

严格来说，Vault 不应该在代码里一次性加载和修改所有子对象；但从领域边界上看，它是最大的数据边界。

2.11.3 领域服务

有些业务动作不属于单个实体，而是跨多个对象协作，这类应该叫领域服务。

领域服务	操作对象	职责
PathGenerationService	LearningPath、Step、Card、Edge	根据主题或资料生成学习路径
DocumentImportService	LiteratureCard、FleetingCard、PermanentCard、Edge、Path	把外部资料拆成卡片和路径
StepExecutionService	Step、Card、Session	点击步骤后创建卡片和会话上下文
CardPromotionService	Card、Session、Edge、RAG、Cognition	将 fleeting 升级为 permanent，并归档线程
WikiLinkSyncService	Card、Edge	根据 [[WikiLink]] 同步关系边
MasteryAssessmentService	Step、Card、Session、Capability	判断用户是否真正掌握
CognitionAnalysisService	Card、Edge、Session、Profile	生成 Cognition 画像和知识缺口
ResourceGenerationService	Card、Path、Resource、PushRecord	生成讲解文档、题目、导图等资源
RagSyncService	Card、RagDocumentIndex	同步卡片到 RAG
VaultExportService	Vault、Card	导出用户自己的 Markdown 内容

这些服务现在分散在 `API route`、`Agent tools` 和 `core service` 中。后续系统设计可以把它们作为更清晰的领域服务抽出来。

2.11.4 领域事件

领域事件表示“系统里发生了一件有业务意义的事”。

领域事件	发生时机	可能影响
VaultCreated	创建知识库	初始化默认空间
CardCreated	创建卡片	刷新 Galaxy / Dashboard
CardUpdated	保存卡片	同步 WikiLink / RAG
CardPromotedToPermanent	卡片升级永久卡	归档线程、更新 Cognition
CardDeleted	删除卡片	清理 Edge / RAG / Session / UI 选中
ClusterCreated	创建星团	刷新 Galaxy
CardAssignedToCluster	卡片归入星团	更新 Galaxy 分组
EdgeCreated	建立卡片关系	更新图谱和知识缺口
DocumentImported	导入资料完成	新增 literature / fleeting / permanent 卡
LearningPathCreated	路径创建	Learn 展示新任务
StepStarted	步骤开始	创建或打开 Forge 线程
StepCompleted	步骤完成	更新 Path 进度和画像
PathArchived	路径归档	从默认任务列表移除
SessionMessageAdded	新增对话消息	更新会话历史和画像输入
SessionArchived	会话归档	阻止继续写入
RagIndexRequested	请求索引	创建或更新 RAG 状态
RagIndexCompleted	索引完成	AI 可召回该卡
RagIndexFailed	索引失败	生成知识缺口或提示
ProfileUpdated	画像更新	影响路径推荐和资源推送
ResourcePushed	资源推送	用户收到推荐

当前代码主要通过 `React Query invalidate`、`SSE`、数据库更新来间接处理这些事件。文档层面应该把它们命名出来。

2.11.5 过程对象

过程对象不是最终内容，但它描述一次业务过程。

过程对象	含义
DocumentImportJob	一次资料导入过程
DocumentChunk	资料被切分后的片段
ExtractedConcept	从资料中抽出的核心概念，通常变成 permanent

过程对象	含义
ExtractedFleeting	从资料中抽出的细节、疑问、例子，通常变成 <code>fleeting</code>
ImportResult	导入结果统计，如 <code>permanent / fleeting / literature / edges</code> 数量
AssessmentResult	一次评估结果
AssessmentQuestion	评估中的题目
FeynmanAssessment	费曼式掌握评估
ResourceGenerationJob	一次资源生成过程
RagIndexJob	一次 RAG 索引任务
OperationConfirmation	高风险操作前的确认

这些对象很多现在没有表，而是藏在 JSON、工具返回值或前端状态里。它们仍然是领域设计中应该承认的对象。

2.11.6 Agent 与工具对象

AXIOM 的 AI 不是一个纯文本输入框，因此 Agent 相关对象也应该进入领域模型，但要和用户知识对象分开。

对象	含义
OracleAgent	主对话与协调者
ProfileAgent	画像分析 Agent
ForgeAgent	卡片打磨和资源生成 Agent
GuideAgent	路径规划 Agent
AssessAgent	评估 Agent
AgentRole	Agent 的角色值对象
ToolDefinition	工具定义
ToolContract	工具风险、能力和边界
ToolCall	一次工具调用
ToolResult	工具返回结果
ToolRisk	<code>read / write / destructive / network / llm</code> 等风险等级
AgentAuditLog	工具、模型、拦截、错误的审计记录

这些对象不是 Card、Path 的替代物。它们属于“AI 执行层”，服务于学习领域对象。

2.11.7 学习引导对象

还有一组对象不是数据库主表，但它们描述“怎么教用户”。

对象	含义
LearningPhase	<code>check / motivation / assessment / generate / learn / verify</code>

对象	含义
TeachingMethod	analogy / example / contrast / formal / visual / socratic 等
LearningStrategy	某阶段采用什么教学方法
UserResponse	用户是否理解、尝试次数、困惑点、aha moment
LearningPattern	对用户有效的学习模式
ExplanationPattern	哪些解释方式有效
ExamplePattern	用户偏好的例子类型
RemedialPattern	补救学习策略

这些对象是 AXIOM 从“聊天工具”变成“学习系统”的关键。当前代码里已经有类型定义，但还没有完全产品化。

3. 数据库模型到领域对象映射

Prisma model	领域对象	说明
user	User	产品用户
account	AuthAccount	登录方式和凭据
session	AuthSession	登录态
verification	VerificationToken	验证凭据
vault	Vault	知识库 / 数据边界
card	Card / FleetingCard / LiteratureCard / PermanentCard	卡片及三种子类型
cluster	Cluster	星团分组
edge	Edge / WikiLinkEdge / PrerequisiteEdge	卡片关系
ragDocumentIndex	RagDocumentIndex	RAG 索引状态
learningPath	LearningPath	学习路径 / 任务组
learningPathStep	LearningPathStep	学习步骤 / 单个任务
learningSession	LearningSession / CardThread / PathStepThread	Forge 学习会话
learningMessage	LearningMessage	会话消息
vaultMemory	VaultMemory	长期记忆
vaultCapability	VaultCapability	概念能力状态
vaultSkill	VaultSkill	用户技能
EducationProfileHistory	EducationProfileSnapshot	教育画像历史
PathAdjustmentHistory	PathAdjustmentRecord	路径调整历史
PushRecord	PushRecord	资源推送记录
agentSession	AgentRuntimeSession	旧 Agent 会话持久化

Prisma model	领域对象	说明
agentAuditLog	AgentAuditLog	Agent 审计事件

4. 核心关系图

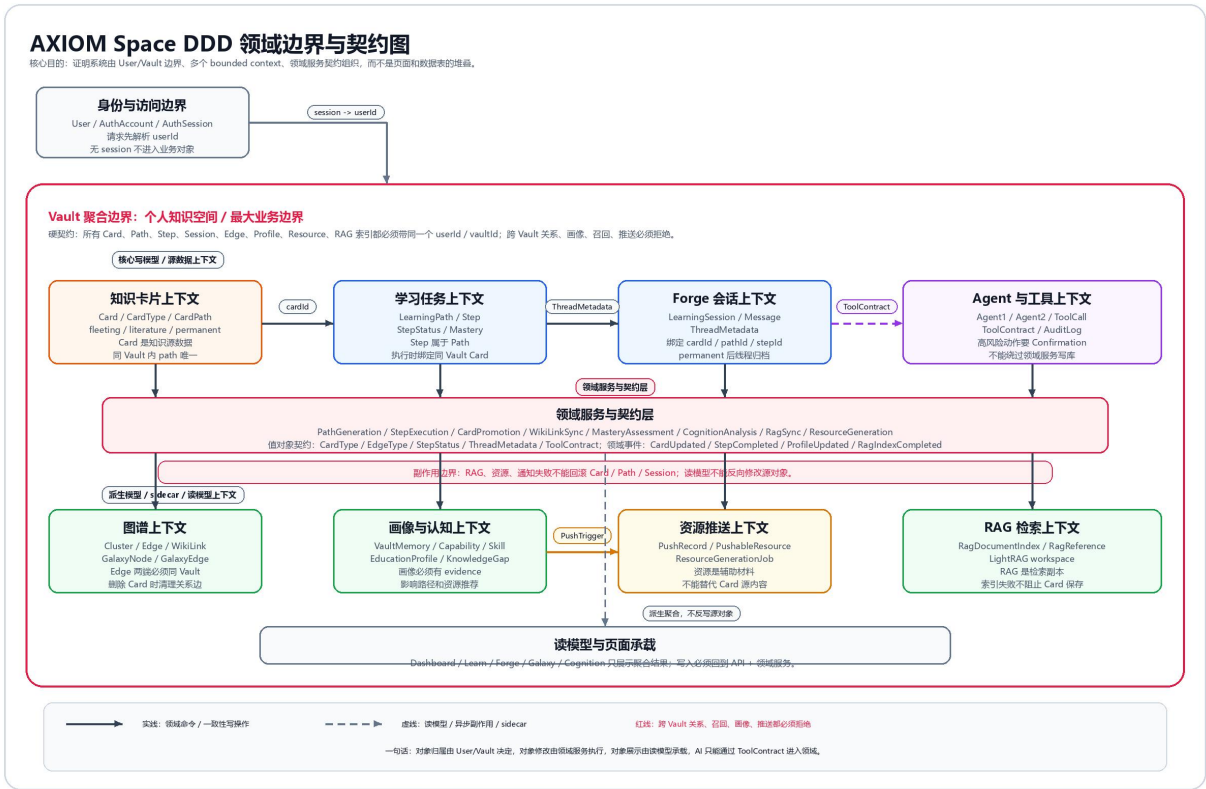


图 1 AXIOM Space DDD 领域边界与契约图

这张图表达的是 DDD 视角下的组织架构：User / Vault 先确定数据归属边界，Card、LearningPath、LearningSession、Graph、Cognition、Resource、RAG、Agent 分别是不同的 bounded context。它们之间不能随意互相改数据，必须通过领域服务、值对象契约、领域事件和工具契约发生协作。

结构示意

```
User
-> AuthAccount
-> AuthSession
-> Vault
-> Card
  -> CardType: fleeting / literature / permanent
  -> CardPath
  -> CardTags
  -> ClusterMembership
  -> Edge(source / target)
  -> RagDocumentIndex
-> Cluster
  -> Card[]
-> LearningPath
  -> LearningPathStep
    -> Card?
  -> PathAdjustmentHistory
```

```
-> LearningSession
  -> ThreadMetadata(cardId/pathId/stepId/threadStatus)
  -> LearningMessage[]
-> VaultMemory
-> VaultCapability
-> VaultSkill
-> EducationProfileHistory
-> PushRecord
-> AgentSession
```

再换成产品语言：

- User 拥有一个或多个 Vault。
- Vault 是知识空间。
- Card 是知识空间里的基本内容。
- Card 分为 fleeting、literature、permanent 三类。
- Cluster 是 Card 的星团分组。
- Edge 是 Card 之间的关系。
- LearningPath 是学习任务组。
- LearningPathStep 是任务组里的一个任务。
- Step 可以绑定 Card。
- LearningSession 是 Forge 对话线程。
- Session 可以绑定 Card、Path、Step。
- Message 是 Session 里的对话消息。
- VaultMemory、Capability、Skill、Profile 是系统对用户学习状态的认识。
- PushRecord 是系统给用户推荐资源的记录。
- RagDocumentIndex 是 Card 进入 RAG 的索引状态。

5. 卡片对象的完整拆解

5.1 Card 是基础对象

Card 的基础字段：

字段	领域含义
id	卡片唯一 ID
vaultId	属于哪个知识库
clusterId	属于哪个星团，可为空
path	在 Vault 中的路径
content	Markdown 内容
type	卡片类型
title	标题

字段	领域含义
tags	标签
createdAt	创建时间
updatedAt	更新时间

Card 的基本契约：

- 必须属于一个 Vault。
- 同一个 Vault 内 path 唯一。
- 可以没有 Cluster。
- 可以没有 PathStep 绑定。
- 可以有多条入边和出边。
- 可以有 RAG 索引状态。

5.2 三类卡片

类型	中文	核心含义	常见来源	常见下一步
fleeting	灵感卡	临时想法、疑问、学习中的不稳定理解	主题生成、Step 执行、用户随手写、从 literature 抽取	进入 Forge 打磨，补定义/例子/关联/应用
literature	文献卡	外部资料、文章、课程、网页、视频、插件采集内容	导入资料、后续浏览器插件、资源生成	拆成 fleeting / permanent，作为引用来源
permanent	永久卡	用户已经整理过、可长期复用的知识	Forge 升级、资料导入出的核心概念	参与 Cognition 统计、RAG 召回、路径规划

三类卡片不是三个互斥空间，而是一条学习沉淀链路：

结构示意

literature 提供资料
-> fleeting 承载临时理解
-> permanent 形成稳定知识

但它们也可以独立存在。例如：

- 用户可以直接创建 fleeting。
- 导入资料时可以直接产生 permanent。
- literature 不一定立刻拆解完成。

5.3 星团分组

Cluster 是卡片的领域分组。

它不是卡片类型，也不是学习路径。

结构示意

Cluster：数据结构

```
-> permanent: 栈
-> permanent: 队列
-> fleeting: 为什么递归能用栈理解
-> literature: 数据结构课程笔记
```

Cluster 的契约:

- Cluster 必须属于 Vault。
- 一个 Card 最多直接属于一个 Cluster。
- 删除 Cluster 不删除 Card，只让 Card 失去分组。
- Cluster 可以包含不同类型的 Card。
- Cluster 可以用于 Galaxy 展示、统计和 Cognition 分析。

5.4 卡片与路径的关系

Card 和 LearningPath 不是同一种对象。

Path 是学习任务组织方式。

Card 是知识沉淀内容。

它们通过 Step 关联:

结构示意

```
LearningPath
-> LearningPathStep
  -> cardId
  -> Card
```

一个 Step 可以绑定一张 Card。

同一张 Card 可以被多个 Step 或多个 Path 复用。

例如:

结构示意

```
Card: 递归

Path: 数据结构入门
  Step: 理解递归调用栈 -> Card: 递归

Path: 算法复杂度
  Step: 分析递归算法复杂度 -> Card: 递归
```

5.5 卡片与会话的关系

Forge 的理想会话是围绕 Card 展开的。

Session 通过 metadata 绑定 Card:

结构示意

```
LearningSession.metadata:
  sessionKind: card-thread
  cardId: xxx
```

```
cardTitle: 第一性原理
cardType: fleeting
threadStatus: active
```

如果这张 Card 来自某个学习任务，还会绑定：

结构示意

```
pathId: xxx
stepId: xxx
```

当 Card 升级为 permanent 后：

结构示意

```
Card.type = permanent
LearningSession.metadata.threadStatus = archived
LearningSession.status = completed
```

也就是说，永久卡的旧打磨线程会归档。

5.6 卡片与图谱的关系

Card 在 Galaxy 里表现为 Node。

Card 之间通过 Edge 连接。

Edge 可以来自：

- 用户或 AI 显式创建关系。
- 导入资料时抽取关系。
- 学习路径生成时建立前置关系。
- Markdown 中的 `[[WikiLink]]` 自动同步。

5.7 卡片与 RAG 的关系

Card 是源数据。

RAG 是检索副本。

结构示意

```
Card.content
-> formatCardForRag
-> LightRAG workspace
-> RagDocumentIndex
```

Card 可以保存成功，但 RAG 索引失败。

所以卡片对象和 RAG 对象必须分开看。

6. 学习路径对象的完整拆解

6.1 LearningPath 是任务组

LearningPath 字段：

字段	领域含义
id	路径 ID

字段	领域含义
userId	所属用户
vaultId	所属 Vault
name	路径名称
topic	学习主题
description	路径说明
difficulty	难度
totalSteps	总步骤数
doneSteps	已完成步骤数
status	状态
source	来源

Path 的状态：

状态	含义
active	当前可继续学习
completed	已完成
archived	已归档

Path 的来源：

来源	含义
ai	AI 根据主题生成
graph	从已有知识图谱推导
manual	用户手动创建
import-document	从资料导入生成，建议后续显式化
unassigned	未归入正式路径的任务视图，当前前端有派生概念

6.2 LearningPathStep 是任务

Step 字段：

字段	领域含义
id	步骤 ID
pathId	所属 Path
cardId	绑定 Card
order	顺序

字段	领域含义
title	任务标题
description	任务说明
concept	对应概念
chapter	所属章节或组
status	任务状态
mastery	掌握度
estimatedMinutes	预计时间
prerequisites	前置依赖

Step 状态：

状态	含义
locked	前置未满足，暂时不可学
available	可以开始
learning	正在学习
completed	已完成
mastered	已掌握

Step 的关键契约：

- Step 必须属于 Path。
- Step 可以没有 Card，但执行时必须创建或绑定 Card。
- Step 的 Card 必须属于同一个 Vault。
- Step 完成后要推动 Path 进度变化。
- Step 的 mastery 不能只等于内容长度，应该逐步接入评估信号。

6.3 PathAdjustment 是路径变化记录

PathAdjustmentHistory 记录路径为什么被调整。

它不是路径本身，而是路径演化的历史。

典型触发：

- 会话结束。
- 评估结果。
- 用户手动反馈。

典型变化：

- 增加复习步骤。

- 跳过已掌握内容。
- 调整难度。
- 增加练习。
- 推荐休息或放缓节奏。

7. Forge 会话对象的完整拆解

7.1 LearningSession

LearningSession 是对话线程。

它可能有三种语义：

类型	含义
conversation	普通对话
card-thread	围绕一张 Card 的打磨线程
path-step-thread	围绕某条 Path 的某个 Step 的学习线程

当前这些类型主要藏在 metadata 里，应该被显式建模。

建议的 ThreadMetadata:

代码示例 · TYPESCRIPT

```
type SessionKind = 'conversation' | 'card-thread' | 'path-step-thread'
type ThreadStatus = 'active' | 'archived'

interface ThreadMetadata {
  sessionKind?: SessionKind
  cardId?: string
  cardTitle?: string
  cardType?: 'fleeting' | 'literature' | 'permanent'
  threadStatus?: ThreadStatus
  pathId?: string
  pathTitle?: string
  stepId?: string
  stepTitle?: string
  archivedAt?: string
}
```

7.2 LearningMessage

LearningMessage 是 Session 下的消息。

角色包括：

role	含义
system	系统消息
user	用户消息
assistant	AI 回复
tool_result	工具结果

Message 的 metadata 可以继续承载：

- 工具调用信息。
- RAG 引用。
- 资源生成进度。
- 压缩信息。
- 评估结果。

7.3 AgentAuditLog

AgentAuditLog 是 Agent 的运行审计对象。

它记录：

- 工具调用。
- 安全拦截。
- 重试。
- LLM 调用。
- 关键错误。

它不直接出现在用户主界面，但对调试、风控、复盘很重要。

8. 画像、记忆、能力对象

8.1 VaultMemory

VaultMemory 是系统记住的用户偏好、风格、上下文和事实。

分类：

category	含义
preference	用户偏好
style	学习或表达风格
context	当前长期上下文
fact	已确认事实

8.2 VaultCapability

VaultCapability 是用户对某个概念的掌握状态。

字段语义：

字段	含义
concept	概念名
masteryLevel	掌握度

字段	含义
status	known / learning / mastered
weakAreas	薄弱点
strongAreas	强项
accessCount	访问次数

8.3 VaultSkill

VaultSkill 表示用户表现出的技能。

它和 Card 不同：

- Card 是知识内容。
- Skill 是用户能力。

例如：

结构示意

Card：递归
Skill：能用递归解决树遍历问题

8.4 EducationProfile

EducationProfile 是六维学习画像。

维度包括：

维度	含义
depth	理解深度
breadth	覆盖广度
connection	知识连接能力
expression	表达能力
application	应用能力
learning_pace	学习节奏

每个维度不是一个裸分数，而是：

结构示意

score + confidence + evidence

所以画像必须有证据，不应该凭空判断。

8.5 CognitionData

CognitionData 是前端看到的认知反馈聚合对象。

它来自：

- Card 数量和类型分布。
 - Edge 和 Cluster 结构。
 - LearningSession 和 Message。
 - RagDocumentIndex 状态。
 - VaultCapability / EducationProfile 等画像对象。
- CognitionData 不应该单独成为源数据。它是派生结果。

9. 资源与推送对象

9.1 PushRecord

PushRecord 表示一次资源推送。

字段语义：

字段	含义
userId	推给谁
vaultId	基于哪个 Vault
resources	推送了哪些资源
trigger	为什么推送
reason	推送理由
sentAt	推送时间
expiresAt	过期时间
viewedAt	用户是否看过
engagedCount	用户互动次数
feedback	用户反馈

trigger 语义：

trigger	含义
stage_completion	阶段完成后推送
assessment_pass	评估通过后推送
low_dimension	某个画像维度偏低
scheduled	定时推送

9.2 PushableResource

PushableResource 是可以被推给用户的资源。

典型字段：

- resourceId
- type
- title
- topic
- difficulty
- estimatedMinutes
- concepts
- tags

9.3 ResourceType

当前资源类型已经不止一种：

类型	含义
document	讲解文档
mindmap	思维导图
quiz	练习题
code	代码案例
video	教学视频或动画
svg	图解
diagram	Mermaid 或结构图
docx	Word 文档
pdf	PDF
ppt	演示文稿

这些资源可以服务 Path、Step、Card 或用户画像中的缺口。

10. RAG 对象

10.1 RagDocumentIndex

RagDocumentIndex 是 Card 的检索索引记录。

状态：

状态	含义
pending	等待索引
indexing	正在索引
indexed	已索引
failed	索引失败
disabled	RAG 未启用或该卡不参与

契约：

- 它依附于 Card。
- 它不能替代 Card。
- 它失败时不能阻止 Card 保存。
- 它成功时说明 AI 可以更稳定地召回该卡。

10.2 RagReference

RagReference 是 AI 回复中的引用对象。

它应该能指回：

- 哪个 Vault。
- 哪张 Card。
- 哪个 title。
- 哪种 card type。
- 哪个文件路径。

这样用户才能知道 AI 的回答依据来自哪里。

11. 页面承载对象

页面	承载对象
Dashboard	Vault、DashboardStats、RecentActivity、GrowthPoint
Learn	LearningPath、LearningPathStep、PathAdjustment、PushRecord
Forge	Card、LearningSession、LearningMessage、RagReference、ResourceProgress
Galaxy	Card、Cluster、Edge、GalaxyNode、GalaxyEdge、GalaxyCluster
Cognition	CognitionData、KnowledgeGap、Observation、EducationProfile、VaultCapability

页面只负责承载对象，不负责定义对象。

真正的对象边界仍然在：

- Vault。
- Card。
- Path。
- Step。
- Session。
- Cluster。
- Edge。
- Profile。

12. 当前最需要收紧的契约

12.1 CardType

现在 `card.type` 是字符串。

应该收敛为：

代码示例 · TYPESCRIPT

```
type CardType = 'fleeting' | 'literature' | 'permanent'
```

12.2 EdgeType

现在 `Edge` 类型在不同工具里有些分散。

建议收敛为：

代码示例 · TYPESCRIPT

```
type EdgeType =  
  | 'related'  
  | 'prerequisite'  
  | 'derived'  
  | 'counter'  
  | 'wikilink'  
  | 'suggests'  
  | 'extends'  
  | 'contrast'  
  | 'application'  
  | 'analogy'
```

12.3 PathStatus 与 StepStatus

代码示例 · TYPESCRIPT

```
type LearningPathStatus = 'active' | 'completed' | 'archived'  
  
type LearningStepStatus =  
  | 'locked'  
  | 'available'  
  | 'learning'  
  | 'completed'  
  | 'mastered'
```

12.4 ThreadMetadata

这是当前最应该从 JSON 字符串中抽出来的对象。

代码示例 · TYPESCRIPT

```
type SessionKind = 'conversation' | 'card-thread' | 'path-step-thread'  
type ThreadStatus = 'active' | 'archived'  
  
interface ThreadMetadata {  
  sessionKind?: SessionKind  
  cardId?: string  
  cardTitle?: string  
  cardType?: CardType  
  threadStatus?: ThreadStatus  
  pathId?: string  
  pathTitle?: string  
  stepId?: string  
  stepTitle?: string  
  archivedAt?: string  
}
```

12.5 Profile 与 Capability

画像和能力对象也需要显式契约：

代码示例 · TYPESCRIPT

```
interface DimensionScore {
  score: number
  confidence: number
  evidence: string[]
}

type CapabilityStatus = 'known' | 'learning' | 'mastered'
```

13. 补充对象的边界与契约

前面已经把主干对象说清楚了：Vault、Card、Cluster、Edge、Path、Step、Session、Profile、Resource、RAG。

但如果要把 AXIOM 真的按 DDD 落实，下面这些对象也要承认。它们不一定都要建表，有些是值对象，有些是过程对象，有些是读模型，有些是领域服务的输入输出。关键不是“有没有表”，而是它们有没有清楚的业务边界。

13.1 总边界原则

原则	说明
Vault 是最大业务边界	任何知识、路径、会话、画像、资源、索引都不能跨 Vault 混用
Card 是知识源数据	资料、评估、RAG、推荐、画像都可以引用 Card，但不能替代 Card
Session 是交互过程	Session 记录学习和打磨过程，不应该成为永久知识本身
Profile 是证据结果	画像、能力、技能必须来自证据，不能凭空生成
RAG 是检索副本	RAG 失败不能阻止 Card 保存，RAG 成功也不能反向篡改 Card
ReadModel 只负责展示	Dashboard、Galaxy、Cognition 的聚合数据不能成为新的源数据
Agent 只通过工具改对象	Agent 不能绕过契约直接写数据库或随意跨对象修改

一个对象只要进入领域模型，就至少要回答四个问题：

结构示意

```
它归属于谁？
它负责什么？
它通过什么契约和别人交互？
它绝对不能直接做什么？
```

13.2 评估与测验对象

评估对象属于学习验证边界。它回答的是：

结构示意

```
用户是真的掌握了，还是只是看过、聊过、写过？
```

对象	边界	职责	契约
----	----	----	----

对象	边界	职责	契约
Assessment	Vault / Step / Card	一次评估的定义	必须有评估目标、评估类型、关联对象
AssessmentAttempt	Session / User	用户的一次作答尝试	必须记录 answer、startedAt、finishedAt
AssessmentQuestion	Assessment	题目	必须有题干、类型、评分规则
AssessmentResult	Attempt	评估结果	必须输出 score、pass、feedback、evidence
Rubric	Assessment	评分标准	必须说明什么算通过、什么算缺陷
CriticalGap	Result	暴露出的关键薄弱点	必须能指向 concept、Card 或 Step
QualityCheckRecord	Card / Session	内容质量检查记录	必须说明检查项、结论和建议

评估对象不能越界：

- 不能直接把 Step 改成 mastered，必须通过 MasteryAssessmentService。
- 不能直接修改 Card 内容，只能给出 feedback 或 suggestion。
- 不能凭一次低分永久降低用户画像，只能作为证据进入 Profile 分析。
- 不能只存一个分数，必须保留可解释的证据。

建议契约：

代码示例 • TYPESCRIPT

```
type AssessmentType = 'quiz' | 'feynman' | 'code' | 'debate' | 'quality-check'

interface AssessmentResult {
  assessmentId: string
  attemptId: string
  score: number
  passed: boolean
  feedback: string
  criticalGaps: CriticalGap[]
  evidence: string[]
}
```

13.3 文档导入对象

文档导入对象属于资料转知识边界。它回答的是：

结构示意

一份外部资料如何进入个人知识库，并被拆成可学习、可连接、可复用的对象？

对象	边界	职责	契约
ImportedDocument	Vault	一次被导入的完整资料	必须有来源、标题、内容 hash
SourceDocument	外部来源	原始网页、PDF、视频稿、笔记	必须保留 sourceUrl 或 sourceName
DocumentChunk	ImportJob	切分后的片段	必须能回溯到原文位置
ExtractedConcept	ImportJob	抽出的核心概念	可以转成 permanent 或 Step concept
ExtractedFleeting	ImportJob	抽出的疑问、例子、临时理解	可以转成 fleeting

对象	边界	职责	契约
ExtractedRelation	ImportJob	概念之间的关系	可以转成 Edge
ImportBatch	Vault	一批导入任务	必须记录成功、失败、跳过
ImportStats	ImportResult	导入统计	输出生成了多少卡片、边、路径
SourceCitation	Card / Resource	引用来源	必须能回到原始资料

文档导入不能越界：

- 不能把外部资料直接等同于 permanent 知识。
- 不能生成没有来源的 literature 卡。
- 不能创建跨 Vault 的 Card 或 Edge。
- 不能覆盖用户已有 Card，只能新建、合并或给出冲突提示。

建议契约：

代码示例 · TYPESCRIPT

```
interface ImportResult {
  literatureCardId?: string
  createdCardIds: string[]
  createdEdgeIds: string[]
  createdPathId?: string
  skippedItems: string[]
  errors: string[]
}
```

13.4 WikiLink 与链接解析对象

WikiLink 对象属于知识连接边界。它回答的是：

结构示意

Markdown 里的 [[概念]] 如何变成稳定、可维护的知识关系？

对象	边界	职责	契约
WikiLink	CardContent	Markdown 中的 [[title]]	必须记录 rawText、targetTitle
ResolvedWikiLink	Vault	已找到目标 Card 的链接	必须有 sourceCardId、targetCardId
DanglingLink	Vault	暂时找不到目标的链接	必须保留 targetTitle
IncomingLink	Card	指向当前 Card 的链接	从 Edge 或解析结果派生
OutgoingLink	Card	当前 Card 指向外部的链接	从内容解析
LinkSyncResult	Card	同步结果	输出新增、删除、悬空链接

WikiLink 不能越界：

- 不能跨 Vault 自动解析。
- 不能因为目标不存在就静默创建 permanent 卡。
- 不能把用户手写的普通文本误认为关系。

- 不能让 Edge 和 Markdown 内容长期不一致。

建议契约：

代码示例 · TYPESCRIPT

```
interface LinkSyncResult {
  sourceCardId: string
  createdEdges: string[]
  removedEdges: string[]
  danglingLinks: string[]
}
```

13.5 卡片质量与升级对象

卡片质量对象属于知识沉淀边界。它回答的是：

结构示意

一张 fleeting 卡什么时候有资格成为 permanent 卡？

对象	边界	职责	契约
CardSection	CardContent	定义、例子、关联、应用等结构段落	必须能从 Markdown 中识别或生成
CardQualityScore	Card	卡片质量评分	必须包含完整性、清晰度、连接度
PromotionCriteria	CardType	升级规则	说明从 fleeting 到 permanent 的最低条件
PromotionAttempt	Card / Session	一次升级尝试	记录是否成功、失败原因
PolishingSuggestion	Card / Session	打磨建议	只能建议，不能强制覆盖用户内容
CardRevision	Card	卡片版本记录	用于追踪内容演化，当前可以先不落表
AIContributionRatio	Card	AI 贡献占比	用于判断是否仍需用户确认

卡片质量对象不能越界：

- 不能只因为 AI 生成得很长就升级 permanent。
- 不能跳过用户确认直接覆盖用户卡片。
- 不能把 literature 直接当成已消化知识。
- 不能把质量评分当成用户掌握度，质量和掌握是两件事。

建议契约：

代码示例 · TYPESCRIPT

```
interface PromotionAttempt {
  cardId: string
  fromType: 'fleeting'
  toType: 'permanent'
  passed: boolean
  missingSections: string[]
  suggestions: string[]
}
```

13.6 认知差距对象

认知差距对象属于画像分析边界。它回答的是：

结构示意

系统从用户的知识结构里看见了哪些缺口、偏差和下一步机会？			
对象	边界	职责	契约
KnowledgeGap	Vault / Profile	一个知识缺口	必须有 gapType、severity、evidence
GapType	KnowledgeGap	缺口类型	no_permanent / isolated / rag_pending / weak_mastery
GapSeverity	KnowledgeGap	严重程度	low / medium / high
CognitiveDimension	Profile	画像维度	depth、breadth、connection、expression、application、pace
ThinkingPattern	Profile	用户思维模式	必须来自会话、卡片或评估证据
Strength	Profile	用户强项	必须说明证据
GrowthEdge	Profile	可成长点	必须能转成建议动作
NextAction	Cognition	下一步建议	必须指向具体 Path、Step、Card 或 Resource

认知差距不能越界：

- 不能把推测当事实。
- 不能没有 evidence。
- 不能直接创建学习路径，只能提出建议或调用路径服务。
- 不能把 Dashboard 统计数字直接等同于认知判断。

13.7 仪表盘读模型对象

仪表盘对象属于展示读模型边界。它回答的是：

结构示意

用户今天打开产品时，应该快速看到自己现在处在什么状态？			
对象	边界	职责	契约
DashboardStats	Vault	汇总统计	只能从 Card、Path、Session、Profile 派生
GrowthPoint	Vault	成长曲线点	必须有时间、指标和值
RecentActivity	User / Vault	最近活动	必须能指回真实对象
ActivityType	Activity	活动类型	card_created / step_completed / session_updated 等
ReviewRate	Vault	复习或回访比例	派生指标，不是源数据
OrphanCardCount	Vault	孤立卡数量	从 Card + Edge 派生

仪表盘读模型不能越界：

- 不能作为源数据被其他核心服务依赖。
- 不能为了展示方便修改领域对象。
- 不能跨 Vault 聚合用户私有数据。

13.8 通知与事件流对象

通知对象属于用户触达边界。它回答的是：

结构示意

系统什么时候、为什么、以什么方式告诉用户有事情发生？

对象	边界	职责	契约
AppNotification	User	一条前端通知	必须有 type、title、message、createdAt
NotificationType	Notification	通知类型	profile / card / skill / graph / resource / system
UnreadCount	User	未读数量	从通知状态派生
EventStreamConnection	Session / Agent	SSE 连接状态	只传递事件，不承载业务真相
NotificationDismissal	User	用户关闭通知	必须记录 notificationId、dismissedAt

通知不能越界：

- 不能替代领域事件。
- 不能因为通知失败就回滚核心业务。
- 不能把敏感内容推到不该出现的页面。

13.9 Agent 安全与确认对象

Agent 安全对象属于执行安全边界。它回答的是：

结构示意

AI 想执行一个动作时，系统如何判断它能不能做、需不需要确认、做完如何审计？

对象	边界	职责	契约
AgentConfirmationRequest	User / Session	高风险操作确认	必须说明 action、risk、payload、expiresAt
ConfirmationToken	Confirmation	确认凭据	必须短期有效、一次性使用
ConfirmationStatus	Confirmation	pending / approved / rejected / expired	状态只能单向变化
ToolRisk	ToolContract	工具风险等级	read / write / destructive / network / llm
AgentAuditEntry	AgentAuditLog	审计条目	必须记录 who、tool、input 摘要、result、time
SecretRedactionRule	Safety	密钥脱敏规则	不允许把 secret 写入日志或模型上下文

对象	边界	职责	契约
ShellHookRule	Safety	命令执行规则	必须区分允许、拒绝、需要确认

Agent 安全对象不能越界：

- Agent 不能绕过 Confirmation 直接执行危险写操作。
- AuditLog 不能保存完整密钥、token、隐私原文。
- ToolResult 不能直接信任，必须经过契约校验。
- Confirmation 不能永久有效。

13.10 Agent 技能对象

AgentSkill 属于 AI 能力边界，VaultSkill 属于用户能力边界。两者必须分开。

结构示意

AgentSkill1 = 系统会什么
VaultSkill1 = 用户会什么

对象	边界	职责	契约
AgentSkill	AgentRuntime	Agent 可调用的能力	必须有名称、触发条件、输入输出
SkillEntry	SkillRegistry	技能注册项	必须能被检索、启用、禁用
SkillSource	SkillEntry	技能来源	built-in / local / imported
SkillSnapshot	Runtime	一次运行时技能快照	防止执行中技能漂移
SkillFilter	Runtime	技能筛选条件	根据任务、模式、风险选择技能
SkillAssessment	Skill	技能适配性判断	输出是否适合当前任务

Agent 技能不能越界：

- 不能和 VaultSkill 混为一谈。
- 不能让未启用或高风险技能自动执行。
- 不能在运行中悄悄改变 ToolContract。

13.11 多 Agent 编排对象

多 Agent 编排对象属于运行时协作边界。它回答的是：

结构示意

多个 Agent 如何分工，而不是互相抢职责？

对象	边界	职责	契约
SubagentRole	Runtime	子 Agent 角色	Oracle / Profile / Forge / Guide / Assess
SubagentMode	Runtime	子 Agent 模式	plan / execute / review / summarize
SubagentStatus	Runtime	当前状态	idle / running / failed / completed

对象	边界	职责	契约
SubagentConfig	Runtime	子 Agent 配置	模型、工具、风险边界
SubagentRunRecord	AgentAudit	一次子 Agent 运行记录	必须记录输入、输出、耗时、失败原因
SubagentEvent	Runtime	编排事件	started / streamed / completed / failed
FlowStep	Orchestration	编排中的一步	必须有前置条件和输出
OrchestrationState	Runtime	当前编排状态	不能成为长期知识源数据

编排对象不能越界：

- ProfileAgent 不能直接写 Card。
- AssessAgent 不能直接改 Step 状态。
- ForgeAgent 不能绕过 CardPromotionService 升级卡片。
- OracleAgent 可以协调，但不能成为所有领域规则的黑箱。

13.12 资源生成与渲染对象

资源生成对象属于学习辅助材料边界。它回答的是：

结构示意

围绕某个学习目标，系统生成了什么材料，它来自哪里，服务哪个对象？

对象	边界	职责	契约
ResourceArtifact	Vault / Card / Step	一份生成资源	必须有 type、title、source、target
ResourceFile	Artifact	资源文件	必须有路径、格式、大小或内容引用
ResourceManifest	ArtifactSet	资源清单	必须列出生成了哪些文件
GeneratedResourceItem	UI / Session	前端展示项	必须能指向真实 Artifact 或 Card
HyperFramesScene	Render	可视化场景	只负责表达，不负责知识真相
VideoGenerationResult	Render	视频生成结果	必须说明成功、失败、输出文件
RenderOptions	Render	渲染参数	不应污染领域对象
GuardrailReport	Safety	生成内容安全检查	必须输出风险和处理建议

资源对象不能越界：

- 不能替代 Card。
- 不能没有来源地进入知识库。
- 不能把渲染配置写进核心 Card 契约。
- 不能把生成失败当成学习失败。

13.13 后台任务对象

后台任务对象属于异步执行边界。它回答的是：

结构示意图

哪些事情不是用户点击后立刻完成，而是要排队、重试、记录状态？

对象	边界	职责	契约
AxiomJob	Queue	通用任务	必须有 name、status、payload、createdAt
RagIndexCardJob	RAG	索引单张卡	输入 cardId，输出 RagDocumentIndex 状态
RagReindexVaultJob	RAG	重建整个 Vault 索引	必须有范围和进度
DocumentImportJob	Import	导入文档任务	输出 ImportResult
ResourceGenerationJob	Resource	生成资源任务	输出 ResourceManifest
JobStatus	Job	queued / running / succeeded / failed / cancelled	状态必须可追踪
QueueName	Job	任务队列名称	区分 rag、import、resource 等

后台任务不能越界：

- 不能没有幂等策略。
- 不能无限重试。
- 不能静默失败。
- 不能让任务状态替代业务对象状态。

13.14 文件、存储与导出对象

这些对象属于基础设施边界，但会影响 Vault 的可迁移性。

对象	边界	职责	契约
FileEntry	Storage	一个文件条目	必须有 path、type、updatedAt
ReadResult	Storage	读取结果	成功时返回内容，失败时返回原因
WriteResult	Storage	写入结果	必须说明是否成功和最终路径
SearchResult	Storage / Search	搜索结果	必须能指回 Card 或文件
VaultExportPackage	Vault	导出包	必须包含 cards、metadata、manifest
ExportArchive	Export	最终压缩包	必须有格式、路径、校验信息

存储对象不能越界：

- 不能让文件路径成为唯一领域身份。
- 不能把本地文件系统实现泄漏到核心领域层。
- 导出不能包含其他用户或其他 Vault 的数据。

13.15 搜索与推荐对象

搜索推荐对象属于发现边界。它回答的是：

结构示意

系统如何从已有知识、记忆、路径、画像中找出当前最相关的东西？

对象	边界	职责	契约
SearchQuery	Vault	搜索请求	必须有 query、scope、limit
SearchResult	Vault	搜索结果	必须有 targetType、targetId、score、reason
MemorySearchResult	Memory	记忆检索结果	必须说明 category 和 relevance
Recommendation	Profile / Path	推荐项	必须有 target、reason、confidence
RecommendationReason	Recommendation	推荐理由	必须可解释
LearningRecommendation	Learning	学习建议	应指向 Step、Card、Resource 或 Path
SuggestedRelation	Graph	建议关系	需要用户确认或服务校验后才能变 Edge

搜索推荐不能越界：

- 不能把推荐直接当事实。
- 不能未经确认创建强关系。
- 不能跨 Vault 检索。
- 不能只给结果，不给理由。

13.16 对话压缩与记忆沉淀对象

这些对象属于长期上下文边界。它回答的是：

结构示意

一段对话里哪些内容应该被保留，哪些应该被压缩，哪些应该沉淀为记忆？

对象	边界	职责	契约
Checkpoint	Session	会话检查点	记录压缩前后的上下文
ReviewableMessage	Session	值得复盘的消息	必须有原因
FlushableMessage	Session	可被沉淀的消息	可以转为 VaultMemory
SummarizedMemory	Memory	压缩后的记忆	必须保留来源
CompressionConfig	Runtime	压缩配置	只影响运行时
CompressResult	Runtime	压缩结果	输出摘要、保留项、丢弃项
DialogueContext	Session	当前对话上下文	不等于永久记忆

压缩对象不能越界：

- 不能把用户一句临时表达直接当长期事实。
- 不能丢失关键决策和来源。
- 不能让压缩摘要覆盖原始 Card。

13.17 模型配置与外部工具对象

这些对象属于系统运行边界，不属于学习核心领域，但它们决定系统能力。

对象	边界	职责	契约
ModelConfig	AI	模型配置	provider、model、temperature、limits
ResolvedModelConfig	Runtime	运行时最终配置	必须可追踪来源
AIProviderConfig	AI	Provider 配置	不应暴露 secret
OracleProfile	Agent	主 Agent 行为配置	不能直接替代用户画像
LLMUsageRecord	Billing / Audit	模型使用记录	记录 token、cost、model
CredentialPool	Security	凭据池	必须有权限和轮换策略
MCPServerConfig	Connector	外部工具服务配置	必须说明可用工具和风险
MCPToolDefinition	Connector	外部工具定义	必须转成 ToolContract 才能执行
ExternalConnector	Connector	外部数据源或工具	必须声明读写范围

模型和外部工具不能越界：

- 不能把 provider 配置写进领域对象。
- 不能让外部工具绕过 ToolContract。
- 不能把外部返回结果不加校验地写入 Vault。

13.18 UI 承载与交互状态对象

UI 状态对象属于表现层边界。它回答的是：

结构示意

用户当前看见什么、选中了什么、正在操作什么？

对象	边界	职责	契约
AppMode	UI	当前主模式	dashboard / forge / galaxy / cognition / learn
PanelLayout	UI	面板布局	只影响展示
PanelId	UI	面板身份	不能替代领域对象 ID
GraphLayoutMode	UI	图谱布局模式	不改变 Edge 语义
SelectedNode	UI	当前选中节点	必须能指回 Card
CanvasAction	UI	画布交互动作	通过 API 或 Store 触发
TypeFilter	UI	卡片类型筛选	不改变 CardType
SortMode	UI	排序方式	不改变源数据

UI 对象不能越界：

- 不能直接写 Prisma。
- 不能把展示筛选结果保存成领域事实。
- 不能让 `selected` 状态影响权限判断。

13.19 推荐的限界上下文

为了后续系统设计更清楚，建议把对象按限界上下文理解：

限界上下文	核心对象	不能越界做什么
Identity	User、AuthAccount、AuthSession	不承载学习业务
Vault Knowledge	Vault、Card、Cluster、Edge、WikiLink	不处理 Agent 执行细节
Learning Task	LearningPath、Step、Assessment、PathAdjustment	不直接篡改 Card 内容
Forge Thread	Session、Message、ThreadMetadata	不替代永久知识
Cognition Profile	Memory、Capability、Skill、Profile、KnowledgeGap	不凭空生成判断
Resource	ResourceArtifact、PushRecord、Manifest	不替代 Card
RAG Search	RagDocumentIndex、RagReference、SearchResult	不作为源数据
Agent Runtime	AgentSkill、ToolCall、Confirmation、AuditLog	不绕过领域服务写对象
Read Model	DashboardStats、CognitionData、GalaxyNode	不成为业务真相

这张表后续可以直接作为系统设计和模块边界的基础。

14. 对象级验收清单

14.1 卡片对象

- Card 必须属于 Vault。
- Card 必须有明确 type。
- type 只能是 `fleeting` / `literature` / `permanent`。
- Card 可以属于 Cluster。
- Card 可以不属于 Cluster。
- Card 可以绑定 Step。
- Card 可以不绑定 Step。
- Card 可以有 Edge。
- Card 可以没有 Edge。
- Card 可以有 RAG 状态。
- `permanent` 卡升级后，绑定活动线程必须归档。

14.2 星团对象

- Cluster 必须属于 Vault。
- Cluster 可以包含三类 Card。
- 删除 Cluster 不删除 Card。
- Card 移出 Cluster 后仍然存在。

14.3 路径对象

- Path 必须属于 User 和 Vault。
- Path 下有多个 Step。
- Step 有顺序。
- Step 可以绑定 Card。
- Path 进度来自 Step 状态。
- Path 归档后不出现在默认 active 列表。
- 删除 Path 不应误删已经沉淀的 Card。

14.4 会话对象

- Session 必须属于 User。
- 学习 Session 应属于 Vault。
- Card Thread 必须绑定 Card。
- Path Step Thread 应绑定 Path 和 Step。
- archived 线程不能继续写入新学习消息。
- Message 必须属于 Session。

14.5 图谱对象

- Edge 两端必须是同一 Vault 的 Card。
- 删除 Card 时相关 Edge 必须清理。
- WikiLink 变化后 Edge 要同步。
- GalaxyNode 来自 Card，不是独立源数据。

14.6 画像对象

- EducationProfile 必须有证据。
- CognitionData 是派生结果，不是源数据。
- VaultCapability 必须绑定具体 concept。
- VaultSkill 必须有 evidence 或 source。

15. 总结

AXIOM 的领域对象不是只有页面上的 Learn、Forge、Galaxy。

更准确的结构是：

结构示意图

```
User
-> Vault
  -> CardSystem
    -> FleetingCard / LiteratureCard / PermanentCard
    -> Cluster
    -> Edge
    -> RagDocumentIndex
  -> LearningTaskSystem
    -> LearningPath
    -> LearningPathStep
    -> PathAdjustment
  -> ForgeThreadSystem
    -> LearningSession
    -> LearningMessage
    -> ThreadMetadata
  -> CognitionSystem
    -> VaultMemory
    -> VaultCapability
    -> VaultSkill
    -> EducationProfile
    -> KnowledgeGap
  -> ResourceSystem
    -> PushRecord
    -> PushableResource
    -> ResourceGeneration
```

其中最核心的闭环仍然是：

结构示意图

```
Path 组织任务
Step 推动行动
Card 承载知识
Cluster 和 Edge 形成结构
Session 承载 AI 引导
Cognition 从沉淀结果中反馈用户状态
```

后续不管写系统设计、API 契约还是测试用例，都应该围绕这些领域对象展开。

附录 B：开发技术规范真源

开发技术规范

技术栈

技术	版本	作用
Next.js	14.2.5	React 框架，App Router
Hono	4.6.2	后端 API 框架，RPC 类型推导
React Query	5.56.2	数据管理，缓存和乐观更新
Zustand	5.x	前端全局状态管理
TypeScript	5	类型安全
shadcn/ui	-	基于 Radix UI 的组件库
Tailwind CSS	4.x	样式框架
Better Auth	-	用户认证
Prisma	6.x	数据库 ORM

核心原则

1. RPC 是 API 调用的唯一方式

使用 Hono 的 RPC 特性调用 API，像调用本地函数一样调用远程 API。

代码示例 · TYPESCRIPT

```
// ✔ 正确: 使用 RPC
import { client } from '@/lib/api-client'
const res = await client.api.tasks.$get()
const data = await res.json()

// ✘ 禁止: 直接使用 fetch/axios
const res = await fetch('/api/tasks')
```

2. 类型推导是关键

使用 InferRequestType 和 InferResponseType 自动推导类型：

代码示例 · TYPESCRIPT

```
import { InferRequestType, InferResponseType } from 'hono/client'

const $post = client.api.tasks.$post
type PostRequestType = InferRequestType<typeof $post>['json']
type PostResponseType = InferResponseType<typeof $post>['data']
```

3. 自定义 Hook 封装一切

在 hooks/ 中封装 RPC + React Query：

代码示例 · TYPESCRIPT

```
// hooks/use-tasks.ts
export function useTasks() {
  return useQuery({
    queryKey: ['tasks'],
    queryFn: async () => {
      const res = await client.api.tasks.$get()
      return (await res.json()).data
    },
  })
}
```

4. 状态管理三原则

- 全局状态 → Zustand (stores/)
- 服务端数据 → React Query (hooks/ 中封装)
- 局部 UI 状态 → useState / useReducer

5. 错误处理统一

代码示例 · TYPESCRIPT

```
// server/api/error.ts 中定义
throw new ApiError({ code: 404, message: '资源不存在' })

// app 层: app.onError(handleError) 统一捕获
```

必遵规范

项目结构

目录	用途
server/api/	Hono API 路由
server/api/routes/	具体路由模块
server/api/error.ts	错误处理
server/api/validator.ts	Zod 校验器
hooks/use-*.ts	自定义 Hook
stores/*.ts	Zustand Store
components/	React 组件
lib/	工具库

React Query 规范

代码示例 · TYPESCRIPT

```
// useQuery: 必须处理 isLoading + error
const { data, isLoading, error } = useQuery({ ... })
if (isLoading) return <div>加载中...</div>
if (error) return <div>加载失败</div>

// useMutation: 必须处理 onSuccess + onError + invalidateQueries
const mutation = useMutation({
  mutationFn: ...,
```

```
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ['tasks'] })
  toast.success('成功')
},
onError: (e) => toast.error('失败: ' + e.message),
})
```

禁止事项

技术	替代方案
fetch / axios / ky	Hono RPC
useState + useEffect 管理数据	React Query
Redux / SWR	React Query
CSS Modules / Styled Components	Tailwind CSS
Ant Design / Material-UI	shadcn/ui

开发流程

添加新功能的标准 4 步：



附录 C：产品创新与设计决策索引

产品创新与方案取舍的完整证据见 [10-产品创新与设计决策全记录](#)。该文档逐篇保留 2 号目录的 10 份原始记录，并与本说明书建立如下对应关系：

设计决策记录	本说明书中的实现责任
01 技术架构说明	第 2、5、9、10 章：分层、事实源、集成与工程边界
02 自研垂直领域 Agent	第 3、4、6、7 章：运行闭环、Harness、画像与评估
03 Web 工作台优先	第 9 章：页面职责、状态管理与系统集成
04 知识图谱创新	第 5、8 章：Card / Edge / Cluster 与认知投影
05 RAG 增强体验	第 5、8、10 章：事实/索引分离、召回、引用与降级
06 清晰、准确、必要	第 3、5、7、10 章：卡片准入、证据与可信边界
07 画像学习闭环	第 3、6、7 章：六维画像、策略编译与教学反馈
08 资源推送机制	第 7、9、10 章：候选发现、用户确认、状态与审计
09 根目录结构规则	第 9、10 章：工程组织、产物治理与可维护性
10 学习效果评估	第 3、6、7、10 章：费曼评估、卡片审核与反馈闭环

资料真源

- docs/01-SDD-文档驱动开发流程/06-DDD-对象模型与契约.md
- docs/01-SDD-文档驱动开发流程/开发技术规范.md
- [10-产品创新与设计决策全记录](#)

— 文档结束 —